



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Functional programs execute by rewriting	
Topic Code	-	
Unit/Module No. & Name	13	Operational Semantics
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-6
2.0	Meaning And Definition	7-9
3.0	Nature Or Type	10-12
4.0	Examples And Case Studies	13-15
5.0	Keywords And Touch Vocabulary	16



OBJECTIVE

1. The main objective of this study material is to help MCA students understand how functional programming programs are executed using rewriting techniques.
2. Functional programming follows a mathematical approach to computation, where programs are treated as expressions rather than sequences of instructions.
3. This objective helps students recognize the difference between functional programming and imperative programming, where variables and memory states are frequently updated.
4. Another objective is to explain the concept of **expression rewriting** as a method of computation.
5. By learning how expressions are simplified step by step using predefined rules, students can understand how functional programs reach a final result known as the normal form.
6. This approach is important for understanding program correctness and predictability.
7. This study material also aims to introduce key concepts, including substitution, reduction, and term rewriting systems.
8. These concepts form the theoretical foundation of functional programming languages and are essential for advanced topics such as compiler design and optimization.
9. An additional objective is to help students understand the role of rewriting in real-world applications.
10. Rewriting techniques are used in compiler and database query optimization, as well as symbolic computation. By studying these applications, MCA students can connect theoretical concepts with practical systems.
11. Finally, this material prepares students for advanced computing subjects. Understanding rewriting models helps students analyze programs mathematically, avoid side effects, and design efficient algorithms.
12. These skills are valuable for higher studies and professional software development in areas such as compilers, databases, and functional language design.

1.0 INTRODUCTION

Functional programming is a programming paradigm that is based on mathematical functions. Unlike imperative programming, it does not focus on changing variables or updating memory states. Instead, it treats programs as **expressions** that are evaluated step by step. Each step follows well-defined rules, and the meaning of the program remains the same throughout execution.

In functional programming, computation is explained using **rewriting**. Rewriting means replacing parts of an expression with simpler expressions until a final result is obtained. This final result is called the **normal form**. This model of execution is clear, predictable, and free from side effects, which makes it very useful for reasoning about programs.

For MCA students, understanding functional execution through rewriting is important. It forms the theoretical foundation for subjects such as compiler design, query optimization, and programming language semantics. This chapter introduces the rewriting model and explains how functional programs are evaluated without mutable state.

1.1 Functional Programming Versus Imperative Programming

Imperative programming focuses on **how** a task is performed. It uses variables, assignments, loops, and memory updates. The program state changes over time as instructions are executed. For example, a variable may start with one value and be updated repeatedly using assignment statements.

Functional programming, on the other hand, focuses on **what** the result of a computation is. It avoids changing variables. Once a value is created, it is not modified. Instead of updating state, new expressions are created and evaluated. This makes functional programs easier to analyze and reason about.

This difference is important for MCA students because many modern systems combine both paradigms. Understanding functional execution helps students design safer and more predictable programs.

1.2 Execution of Programs Using Rewriting

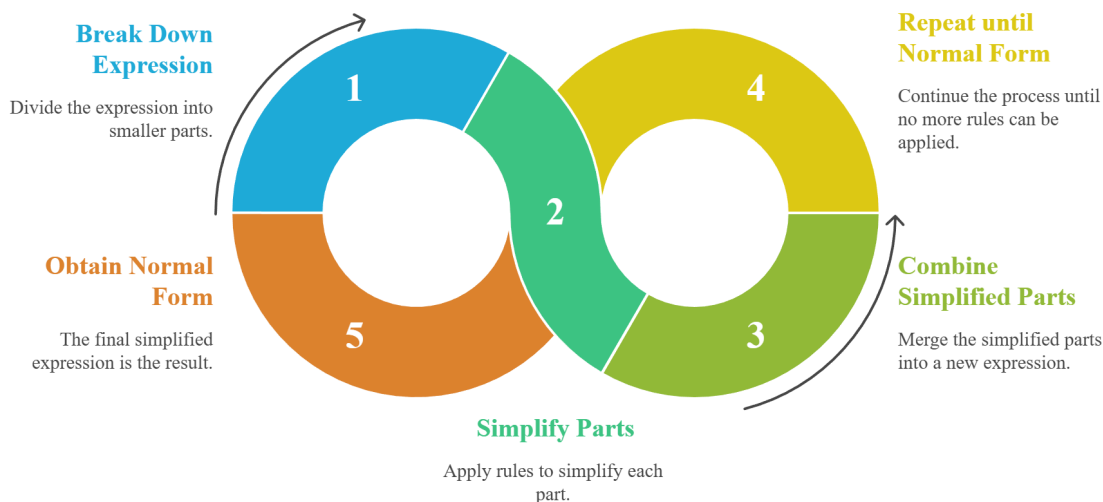
In functional programming, execution happens through **rewriting of expressions**. An expression is broken into smaller parts. Each part is simplified using predefined rules. The process continues until no more rules can be applied.

For example, an arithmetic expression is simplified step by step. Each rewrite preserves the meaning of the expression. There is no use of memory updates or variable reassignment. This makes the execution model purely mathematical.

Rewriting guarantees that even if different rewrite paths are followed, they will lead to the same result. This property is known as **confluence**. It ensures correctness and reliability in functional computation.

Fig. 1

Functional Program Execution Cycle



This diagram explains the functional program execution cycle, showing how expressions are broken down, simplified, recombined, and repeatedly processed until a final normal form is achieved.

1.3 Importance of Rewriting Model for MCA Students

The rewriting model is important because it provides a clear and formal way to explain program execution. It helps MCA students understand how compilers analyze and optimize

code. Many compiler optimizations are based on rewriting rules that transform code without changing its meaning.

In database systems, rewriting is used to optimize queries. Logical query expressions are rewritten into more efficient forms before execution. This improves performance without changing results.

By learning the rewriting model, MCA students gain strong analytical skills. These skills are useful for advanced computing topics and professional software development.



2.0 MEANING AND DEFINITION

The meaning and definition of functional programming execution through rewriting help students clearly understand how programs are evaluated without changing variables or memory states. This concept explains computation as a sequence of logical steps that simplify expressions until a final result is reached. Each step follows predefined mathematical rules, ensuring correctness.

In functional programming, execution is not based on instructions that change values. Instead, it is based on transforming expressions. This approach provides a clean and structured way of reasoning about programs. For MCA students, understanding this meaning is essential because it forms the base for compiler theory, programming language design, and database optimization.

2.1 Meaning of Functional Program Execution

Functional program execution means evaluating expressions step by step using rewriting rules. Each expression represents a computation, and evaluation happens by simplifying the expression logically. There are no assignments or updates to variables.

For example, an arithmetic expression is evaluated by replacing smaller parts with their computed values. This process continues until the expression cannot be simplified further. The final simplified expression represents the result of the program.

This meaning helps students understand why functional programs are predictable and easy to test. Since there are no side effects, the same input always produces the same output.

2.2 Definition of Rewriting in Functional Programming

Rewriting can be formally defined as the process of replacing a part of an expression with another expression according to a fixed rule. These rules are defined by the programming language or the mathematical system used.

Each rewrite step preserves the meaning of the original expression. This ensures that the final result is logically correct. Rewriting continues until no rule can be applied. At this stage, the expression is said to be in its normal form.

This definition is important because it explains how functional languages avoid errors caused by changing program states.

2.3 Definition of Normal Form

Normal form is the final and fully simplified result obtained after all possible rewriting steps have been applied to an expression. An expression is said to be in normal form when no rewriting rule can be applied to it anymore. At this stage, computation naturally stops because there is nothing left to simplify. This clearly indicates the completion of program execution.

For example, a numeric value such as 11 is already in normal form because it cannot be reduced or rewritten further. In functional programming, expressions gradually move toward normal form through systematic rewriting. Each rewriting step simplifies a part of the expression while preserving its meaning. When the expression reaches normal form, the program has successfully produced its output.

Understanding normal form is important for MCA students because it helps them recognize when computation has ended correctly. It also supports program verification, where students must ensure that programs terminate properly and produce valid results. In optimization, compilers attempt to rewrite expressions efficiently so that normal form is reached with fewer steps, improving performance.

2.4 Definition of Term Rewriting System

A Term Rewriting System, commonly known as TRS, is a formal mathematical model used to represent computation through rewriting. It consists of three main components: terms, variables, and rewriting rules. Terms represent expressions, variables act as placeholders, and rewriting rules define how one term can be replaced with another.

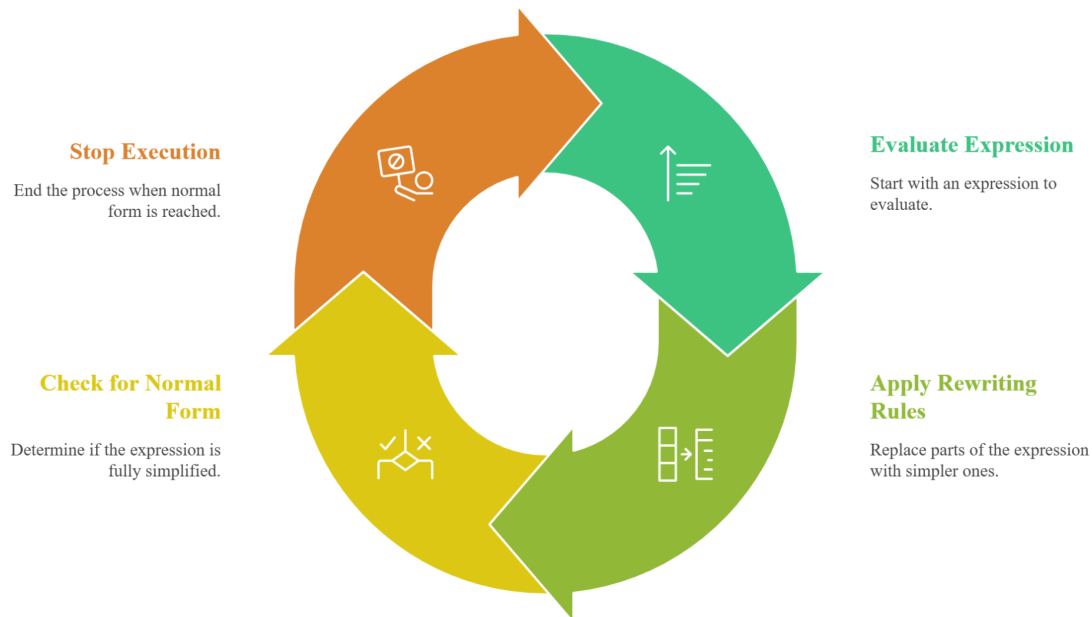
Each rewriting rule in a TRS shows a valid transformation that preserves meaning. Computation proceeds by repeatedly applying these rules to an expression until it reaches normal form. TRS ensures that computation is systematic, logical, and free from ambiguity. This makes it a powerful tool for reasoning about program behavior.

TRS provides a strong mathematical foundation for functional programming execution. It is used to prove important properties such as correctness, termination, and consistency of

programs. For MCA students, understanding TRS helps bridge the gap between programming and formal logic. It also prepares students for advanced topics such as compiler construction, symbolic computation, and automated reasoning systems.

Fig. 2

Functional Program Execution Cycle



This circular flow illustrates functional program execution, starting with expression evaluation, applying rewriting rules, checking for normal form, and stopping execution once full simplification is reached.

3.0 NATURE OR TYPE OF FUNCTIONAL PROGRAM EXECUTION USING REWRITING

The nature of functional program execution explains how computation is carried out without changing variables or memory states. In functional programming, execution is based on rewriting expressions rather than performing step-by-step instructions. Each rewrite step follows strict rules and preserves the meaning of the program. This nature makes functional execution predictable, reliable, and mathematically sound.

Functional execution treats programs as expression trees. These expressions are simplified gradually until they reach normal form. The absence of mutable state ensures that results do not depend on execution order or external changes. This property is especially important for reasoning, optimization, and correctness.

3.1 Nature of Substitution

Substitution is a fundamental operation in functional programming execution. It occurs when a function is applied to an argument. The variable in the function definition is replaced by the actual value provided. This replacement is done directly in the expression without changing any stored data.

For example, if a function defines a rule such as adding one to a value, applying it to a number replaces the variable with that number. The resulting expression is then simplified further using rewriting rules. Substitution ensures that computation remains pure and side-effect free.

This nature of substitution helps students understand how functions behave like mathematical mappings rather than procedures with memory changes.

3.2 Nature of Reduction

Reduction refers to a single rewriting step where an expression is simplified according to a rule. Multiple reductions can be applied sequentially to reach the final result. Each reduction moves the expression closer to normal form.

There are different reduction paths, but all valid paths lead to the same result if the rewriting system is well defined. This property ensures consistency in computation. Reduction does not alter meaning; it only simplifies structure.

Understanding reduction helps MCA students analyze program execution and identify optimization opportunities in compilers.

3.3 Nature of Confluence

Confluence is a key property of rewriting systems. It means that even if different rewriting paths are followed, they will eventually lead to the same normal form. This guarantees that the final result of a computation is independent of the order of rule application.

Confluence ensures correctness and reliability. Programmers do not need to worry about which part of the expression is rewritten first. The result will always be the same.

This nature is important for compiler optimization and parallel evaluation, where different parts of an expression may be processed independently.

3.4 Nature of Term Rewriting Systems in Computation

Term Rewriting Systems provide a formal structure for rewriting. They define exactly how expressions are transformed. TRS ensures that computation follows well-defined mathematical rules rather than ad-hoc procedures.

The nature of TRS supports formal reasoning about programs. It helps prove properties such as termination and correctness. For MCA students, this nature is essential for understanding compiler theory and program optimization.

TRS also connects functional programming with logic and algebra, strengthening theoretical foundations.

3.5 Nature of Functional Execution in Modern Systems

The rewriting nature of functional execution is widely used in modern systems. Compilers rewrite expressions to generate efficient code. Database systems rewrite queries to improve performance. Symbolic computation systems rewrite formulas to simplify results.

This nature makes functional programming valuable beyond academic study. It provides practical benefits in optimization, parallelism, and correctness.

For MCA students, understanding this nature prepares them for advanced computing topics and real-world system design.



4.0 EXAMPLES AND CASE STUDIES

Examples and case studies help MCA students clearly understand how functional program execution through rewriting works in real situations. They show how expressions are evaluated step by step, how rewriting replaces computation with simplification, and how these ideas are applied in compilers and database systems. These case studies connect theory with practical applications.

4.1 Case Study: Arithmetic Expression Evaluation Using Rewriting

Consider the arithmetic expression

$$4 + (6 + (4 \times (10 \div 2))).$$

In functional execution, this expression is evaluated by rewriting. First, the inner division is simplified. Then multiplication is applied, followed by addition. Each step replaces a subexpression with a simpler one. The expression gradually becomes simpler until it reaches a final value.

No variables are updated, and no memory is changed. Each rewrite step preserves the meaning of the expression. The final result is a number that cannot be simplified further. This final value is the normal form.

This case study shows how functional execution works as a mathematical process and avoids side effects.

4.2 Case Study: Simple Expression Reduction

Consider the expression

$$3 + (2 \times 4).$$

The rewriting process begins by identifying the innermost operation. The multiplication rule is applied first, rewriting 2×4 as 8. The expression then becomes $3 + 8$. Next, the addition rule is applied, rewriting the expression to 11.

At this point, no rewriting rule can be applied. The expression has reached its normal form. The computation is complete.

This case study demonstrates reduction and shows how complex expressions are simplified using a sequence of rewrite steps.

4.3 Case Study: Function Application Using Substitution

Consider a functional expression where a function adds one to its input. When the function is applied to a number, substitution takes place. The variable inside the function is replaced with the actual value.

For example, applying the function to the value 4 replaces the variable with 4 in the expression. The rewritten expression is then simplified to produce the final result.

This case study explains substitution as a key concept in functional execution. It shows how functions behave like mathematical rules rather than procedures that modify memory.

4.4 Case Study: Compiler Optimization Using Rewriting

In an MCA compiler project, functional rewriting is used to optimize code. Expressions written by the programmer are transformed into simpler or faster versions before execution.

For example, repeated calculations can be rewritten into a single expression. Unnecessary intermediate steps are removed. This reduces execution time and improves efficiency.

This case study shows how rewriting is used in compiler design to produce optimized machine code without changing program meaning.

4.5 Case Study: SQL Query Optimization Using Rewriting

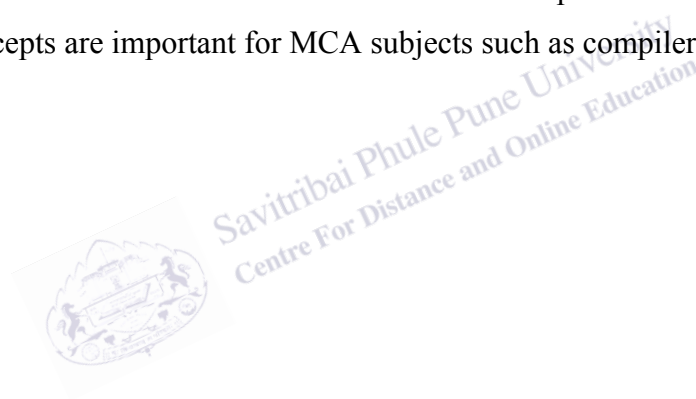
In database systems, queries are often rewritten before execution. For example, the order of join operations can be changed to reduce computation cost.

A query written in one form is transformed into another equivalent form that runs faster. The result remains the same, but performance improves significantly.

This case study shows how rewriting is applied in database management systems. It highlights the importance of rewriting in MCA subjects such as DBMS and data analytics.

1. Functional programming treats programs as **expressions**, not instructions.
2. Execution in functional programming is done using **rewriting**.

3. Rewriting replaces complex expressions with simpler ones step by step.
4. Functional execution does not use variable updates or memory changes.
5. Each rewrite step preserves the meaning of the expression.
6. The final simplified result of rewriting is called **normal form**.
7. Computation ends when no rewriting rule can be applied.
8. **Substitution** replaces a variable with its actual value during function application.
9. **Reduction** is a single step in the rewriting process.
10. Multiple reductions lead an expression toward normal form.
11. **Confluence** ensures that different rewrite paths lead to the same result.
12. Term Rewriting Systems provide a formal model for computation.
13. Rewriting is used in compiler optimization to improve performance.
14. Database systems use rewriting to optimize SQL queries.
15. Functional execution improves correctness and predictability.
16. Rewriting avoids side effects and ensures reliable computation.
17. These concepts are important for MCA subjects such as compilers and DBMS.



5.0 KEYWORDS AND TOUCH VOCABULARY

Keyword	Meaning
Functional Programming	Programming based on mathematical functions
Rewriting	Replacing expressions using rules
Expression	A combination of values and operations
Normal Form	Final result of rewriting
Substitution	Replacing a variable with a value
Reduction	Single rewriting step
Confluence	Property where all rewrite paths give same result
Term	Basic unit in rewriting systems
Rewrite Rule	Rule defining how expressions change
Term Rewriting System	Formal model for rewriting computation
Imperative Programming	Programming with variable updates
Compiler Optimization	Improving code without changing meaning
SQL Query Optimization	Rewriting queries for better performance
Side Effect	Change in state outside expression
Evaluation	Process of computing an expression
Mathematical Model	Logic-based explanation of computation
MCA Project	Academic software project for MCA students